

电控第一次作业 说明文档

项目	内容
姓名	_____
学号	_____
板子	STM32F103C8T6 最小系统板（板载 8 MHz 晶振）
调试器	J-Link（SWD）
完成日期	2026 年 10 月 ____ 日

一、作业内容与工程概况

三题做在同一个 CubeMX 工程里：

- 1. **GPIO**：F103 最小系统板把 PC13 配成推挽输出，写成低电平点亮板载灯（不要在 while(1) 里翻转）。
- 2. **定时器**：用一个定时器的更新中断，周期 1 ms，回调里让全局变量 tick 自增 1，并调用 HAL_IWDG_Refresh 喂狗。
- 3. **看门狗**：IWDG 保持启用、分频和超时都不改，tick 仍每 1 ms 加 1，但去掉回调里的 HAL_IWDG_Refresh。

软件环境

软件	版本 / 说明
STM32CubeMX	6.18.1（固件包 STM32Cube FW_F1 V1.8.7）
工具链	VS Code + CMake 4.4.3 + Ninja + arm-none-eabi-gcc 15.2.1
构建类型	Debug（发放的 CMakeLists.txt 模板，user_folders 为 "Tasks"）
调试	SEGGER Ozone V3.26，打开 build/project.elf
芯片	STM32F103C8Tx（LQFP48）

工程结构（约定：业务代码只写在 Tasks 里，main.c 只在 USER CODE 段里改）

hjnb_666/	
├ Core/	CubeMX 生成：main.c / gpio.c / tim.c / iwdg.c / stm32f1xx_it.c ...
├ Drivers/	CubeMX 生成：HAL 驱动 + CMSIS
├ Tasks/	自己的业务代码（不会被 CubeMX 冲掉）
├ └ inc/	task_led.h task_timer.h task_motor.h
├ └ src/	task_led.c task_timer.c task_motor.c
├ cmake/	CubeMX 生成的 CMake 配置
└ CMakeLists.txt	发放的模板（每次 CubeMX 重新生成后要换回来）

hjjnb_666.ioc	CubeMX 工程配置
build/project.elf	构建产物, Ozone 直接打开它

二、系统时钟与"定时器时钟从哪来"

CubeMX 的 Clock Configuration 页面配置如下 (RCC 里 HSE 选 Crystal/Ceramic Resonator) :

节点	取值	说明
HSE	8 MHz	板载晶振, 作为 PLL 输入
PLL	$HSE \times 9 = 72 \text{ MHz}$	PLL Source Mux 选 HSE
SYSCLK	72 MHz	System Clock Mux 选 PLLCLK
AHB 预分频	$\div 1$	HCLK = 72 MHz
APB1 预分频	$\div 2$	PCLK1 = 36 MHz (CubeMX 时钟树里显示 APB1 peripheral clocks = 36 MHz)
APB2 预分频	$\div 1$	PCLK2 = 72 MHz
APB1 timer clocks	72 MHz	这一行就是 TIM2 的定时器时钟
Flash 延时	2 个等待周期	72 MHz 必须配 FLASH_LATENCY_2

关键点: TIM2 挂在 APB1 上。当 APB1 的预分频系数不等于 1 时, 定时器的时钟是 APB1 时钟的 2 倍, 所以

TIM2 定时器时钟 = PCLK1 $\times$ 2 = 36 MHz $\times$ 2 = 72 MHz

这一点在 CubeMX 时钟树里直接写成了 APB1 timer clocks = 72 MHz, 后面 PSC 就按 72 MHz 来算。

三、第 1 题: GPIO (PC13 点亮板载灯)

CubeMX 配置: System Core $\rightarrow$ GPIO $\rightarrow$ PC13 选 GPIO_Output, 详细设置里

参数	取值
GPIO output level	Low (低电平点亮)
GPIO mode	Output Push Pull
GPIO Pull-up/Pull-down	No pull-up and no pull-down

Maximum output speed	Low
User Label	LED_PC13

生成的代码在 Core/Src/gpio.c 的 MX_GPIO_Init() 里：先
HAL_GPIO_WritePin(LED_PC13_GPIO_Port, LED_PC13_Pin, GPIO_PIN_RESET);，再把
PC13 配成推挽输出。业务动作按作业要求写在 Tasks 里：

```
/* Tasks/src/task_led.c */
#include "task_led.h"

void Task_Led_Init(void)
{
    /* 低电平点亮板载灯（PC13 上的 LED 接 3.3V，所以低电平亮） */
    HAL_GPIO_WritePin(LED_PC13_GPIO_Port, LED_PC13_Pin, GPIO_PIN_RESET);
}
```

现象：下载后板载灯亮起，while(1) 里没有任何翻转代码。

说明：第 1 题时还没有启动定时器（main.c 里 Task_Timer_Init() 被临时注释掉），所以没人喂狗，芯片大约每 2 秒复位一次，灯会闪一下再亮。这是符合预期的现象，拍照要在灯亮的时候拍。

四、第 2 题：定时器 1 ms tick + 喂狗

4.1 CubeMX 配置

参数	取值
TIM2 Clock Source	Internal Clock
Prescaler (PSC)	71
Counter Period (ARR)	999
Counter Mode	Up
auto-reload preload	Disable
NVIC	TIM2 global interrupt 使能

4.2 PSC / ARR 的计算过程

作业给出的公式是 $(PSC + 1) \times (ARR + 1) / \text{定时器时钟} = 0.001 \text{ s}$ 。

- 定时器时钟：由第二节可知，TIM2 的定时器时钟 = **72 MHz**（HSE 8 MHz $\times$ PLL9 = 72 MHz，APB1 $\div 2 = 36 \text{ MHz}$ ，定时器时钟是 APB1 的 2 倍）。
- 先让计数器 1 个数代表 1 μs ：PSC = 72 MHz / 1 MHz - 1 = 71；
- 再让计数器数 1000 个数进一次中断：ARR = 1000 - 1 = 999。

```
更新中断周期 = (PSC + 1) × (ARR + 1) / 定时器时钟
               = (71 + 1) × (999 + 1) / 72 000 000
               = 72 × 1000 / 72 000 000
               = 0.001 s = 1 ms
```

即中断频率 1 kHz，tick 每秒增加 1000。

4.3 回调与中断链路

Tasks/src/task_timer.c 中的实现（全工程只保留这一份更新回调）：

```
/* 作业要求变量名必须叫 tick，且为全局 volatile uint32_t */
volatile uint32_t tick = 0U;

void Task_Timer_Init(void)
{
    HAL_TIM_Base_Start_IT(&htim2);    /* 打开 TIM2 更新中断并启动计数 */
}

/* 更新中断回调：HAL_TIM_IRQHandler() 判断出是 TIM2 的更新事件后调用它 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        tick++;                        /* 每 1 ms 加 1 */
        HAL_IWDG_Refresh(&hiwdg);     /* 喂狗（第 3 题把这一行去掉） */
    }
}
```

中断的调用链是：

```
TIM2 更新事件
→ TIM2_IRQHandler()          (Core/Src/stm32f1xx_it.c, CubeMX 生成)
→ HAL_TIM_IRQHandler(&htim2)  (HAL 驱动，自动判断中断来源)
→ HAL_TIM_PeriodElapsedCallback(&htim2)  (Tasks/src/task_timer.c, 自己实现)
→ tick++ 且 HAL_IWDG_Refresh(&hiwdg)
```

注意：CubeMX 生成的 Core/Src/tim.c 只负责初始化（MX_TIM2_Init()，里面有 PSC=71、ARR=999），它不会再生成同名回调；如果勾选了 Register Callback 导致 main.c 里也出现一份 HAL_TIM_PeriodElapsedCallback，必须删掉那一份，否则链接报重复定义。

4.4 现象与附录截图

下载后程序一直运行，用 Ozone 的 Watched Data（或 Timeline）观察 tick，可以看到它一直增加，每秒大约增加 1000。截图见 附录 图 2。

注意：观察时程序要保持运行，不要停在断点上——CPU 停住时看门狗仍在计数，会导致复位、调试器掉线。

五、第 3 题：看门狗（去掉喂狗）

5.1 IWDG 配置（三题都不要改）

参数	取值
IWDG counter clock prescaler	64
Down-counter reload value	1249
LSI	约 40 kHz（F103 手册典型值）

$$\begin{aligned}\text{超时} &= (\text{Reload} + 1) \times \text{分频} / \text{LSI} \\ &= (1249 + 1) \times 64 / 40\,000 \\ &= 80\,000 / 40\,000 \\ &= 2.0\text{ s}\end{aligned}$$

超时 2 秒远大于 1 ms 的中断周期，所以第 2 题里每 1 ms 喂一次狗，芯片不会复位。

5.2 第 3 题的改动

只改一处：把 Tasks/src/task_timer.c 里的开关

```
#define TASK_IWDG_REFRESH 1U /* 第 2 题：1 = 回调里喂狗 */
```

改成

```
#define TASK_IWDG_REFRESH 0U /* 第 3 题：0 = 不喂狗 */
```

重新编译下载。等价于“去掉回调里的 HAL_IWDG_Refresh”，其它代码（分频、超时、中断周期）一个字都不动。

5.3 现象与附录截图

芯片在最后一次喂狗后约 2 秒被看门狗复位，复位后启动代码会把全局变量清零，于是 tick:

从 0 涨到大约 2000 → 归零 → 再涨到约 2000 → 再归零，不断重复（2000 ms ÷ 1 ms = 约 2000 次）。

用 Ozone 的 Timeline（或连续监视窗口）截图，能看到“涨上去再归零”的过程，见附录图 3。

同样不要停在断点上；复位瞬间调试器可能短暂掉线，重新连接后继续观察即可。

六、遇到的问题与解决过程

1. 一开始 tick 每秒只增加 100 多：检查发现 CubeMX 里 RCC 只开了 HSI、PLL 处于关闭状态，系统时钟是 8 MHz。而代码里 $PSC=71/ARR=999$ 是按 72 MHz 算的，实际周期变成 $72 \times 1000 / 8000000 = 9 \text{ ms}$ 。把 RCC 的 HSE 改成 Crystal/Ceramic Resonator、PLL×9 配到 72 MHz 后，tick 恢复为每秒约 +1000。
2. 链接报 g_pfnVectors 重复定义：发放模板里用 file(GLOB_RECURSE startup_file "*.s") 递归收集启动文件，工程目录里如果放了带 startup_*.s 的备份/临时目录，会被一起编进来。把备份目录移出工程目录后解决。
3. TIM2_IRQHandler 重复定义：CubeMX 启用 TIM2 中断后会在 Core/Src/stm32f1xx_it.c 里生成一份 TIM2_IRQHandler，如果之前自己手写过一份，就会重复。删掉手写的那一份，统一走 HAL_TIM_IRQHandler(&htim2)。
4. Ozone 一暂停就复位：CPU 停在断点时看门狗照常计数，2 秒后复位、调试器掉线。观察 tick 时全程保持运行。
5. CubeMX 重新生成会覆盖 CMakeLists.txt：每次生成后检查是不是发放的模板，被盖掉就换回来；Tasks 目录和 USER CODE 段里的代码不会被冲掉。

七、附录

按作业要求，正文不放图，所有截图和照片统一放在这里。

图 1 第 1 题：PC13 板载灯点亮

（在此插入照片：开发板通电后，板载 LED 处于点亮状态）

图注：STM32F103C8T6 最小系统板，PC13 输出低电平，板载灯点亮。

图 2 第 2 题：tick 的 Ozone 监视 / Timeline 截图（每秒约 +1000）

（在此插入截图：Ozone 运行状态下的 Watched Data 或 Timeline，程序处于运行中，不停在断点上）

图注：TIM2 更新中断周期 1 ms，tick 每秒增加约 1000。注意图中有两处时间戳的数值差，可以看出增长速率。

图 3 第 3 题：tick 的 Ozone Timeline 截图（涨到约 2000 再归零）

（在此插入截图：能看出 `tick` 上升、归零、再上升的循环）

图注：回调里不再喂狗，IWDG 约 2 秒超时复位芯片，复位后全局变量被清零，因此 `tick` 在 0 与约 2000 之间循环。

八、其他说明

- 构建产物 `build/project.elf` 供 Ozone 打开；`build/` 目录不纳入版本管理。
- 代码仓库：https://github.com/_____/（作业要求的公开仓库链接）
- 本文档中第 1、2、3 题的截图分别来自三次不同的下载：第 1 题（不启动定时器、不喂狗）→ 第 2 题（启动定时器并喂狗）→ 第 3 题（启动定时器但不喂狗）。